Week 4

Temporal-Difference Learning

SARSA, Q-learning, Expected SARSA, Double-Q

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
- 2.1 The TD error . . . 5
- 2.2 Tabular TD(0) for prediction . . . 5
- 2.3 n-step TD and the bias–variance dial . . . 5
- 2.4 SARSA: on-policy TD control . . . 6
- 2.5 Q-learning: off-policy TD control . . . 6
- 2.6 Expected SARSA . . . 7
- 2.7 Maximisation bias and Double-Q . . . 7
- 2.8 On-policy vs. off-policy: the cliff-jumping gap . . . 7

3 Algorithms 8

4 Implementation walkthrough 9

5 Empirical results 10
- 5.1 CliffWalking bake-off . . . 10
- 5.2 Taxi-v3 . . . 10

6 Exercises 13

7 Further reading 14

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Derive the one-step temporal-difference (TD) error $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ from the Bellman equation and show that its conditional mean is the Bellman residual for a fixed value estimate.
2. State and analyse the bias–variance trade-off between Monte Carlo, n-step TD, and TD(λ).
3. Implement SARSA (on-policy), Q-learning (off-policy), Expected SARSA, and Double-Q learning from scratch on `CliffWalking-v1` and `Taxi-v3`.
4. Explain why on-policy and off-policy control converge to different policies under a fixed ε-greedy behaviour (the cliff-jumping behaviour gap).
5. Construct a 2-state MDP where Q-learning suffers maximisation bias and show how Double-Q separates selection from evaluation without guaranteeing zero bias.
6. Connect TD-style online updates to incremental policy improvement in a lead-scoring matching system and to off-policy evaluation via self-normalised IS.

1 Motivation and intuition

Monte Carlo methods (Week 3) wait until the end of an episode to learn anything. They are unbiased but high variance, and they are useless for non-episodic problems. Dynamic programming (Week 2) updates after every transition but requires the transition kernel. Temporal-difference learning is the love child of the two: *bootstrap* from a *sampled* successor.

Sutton called this his single happiest research idea. The TD prediction algorithm is on a short list of "ideas you almost regret not having yourself."

Intuition. You are driving home and want to learn how long the rest of the trip takes. Monte Carlo says: wait until you get home, observe the total, update. TD(0) says: *the moment* you exit the motorway you should update your estimate of how long the trip will be from the *previous* landmark, using the new (better) estimate from the motorway exit. You borrow your future self's prediction. As long as the future self is improving, so do you.

A second intuition: TD is online, incremental, and model-free. That makes it the right primitive for systems that update one sample at a time — e.g. a production lead-scoring policy that updates after each conversion outcome lands.

Remark. Throughout this chapter V_π and Q_π refer to the value functions of a policy π. We use Q for the action-value table; tabular control methods estimate $Q^\star$ via samples from the environment.

2 Mathematical formulation

2.1 The TD error

The Bellman equation for V_π reads

$$V_\pi(s) = \mathbb{E}_{\pi,\mathcal{T}}\big[\, r_{t+1} + \gamma V_\pi(s_{t+1}) \,|\, s_t = s\big].$$

Suppose at time t we observe a transition $(s_t, a_t, r_{t+1}, s_{t+1})$ where $a_t \sim \pi(\cdot|s_t)$. Define the *one-step TD target*

$$Y_t \triangleq r_{t+1} + \gamma V(s_{t+1}),$$

and the *TD error*

$$\delta_t \triangleq Y_t - V(s_t) = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

Theorem 2.1.1 (Unbiasedness at the fixed point)**.** *If $V = V_\pi$, then $\mathbb{E}[\delta_t \,|\, s_t] = 0$ for every state visited under π.*

Proof. By linearity of expectation and the Bellman equation,

$$\mathbb{E}[\delta_t \,|\, s_t] = \mathbb{E}[r_{t+1} \,|\, s_t] + \gamma \mathbb{E}[V(s_{t+1}) \,|\, s_t] - V(s_t) = V_\pi(s_t) - V(s_t) = 0.$$

Outside the fixed point, the vector of conditional mean TD errors is the Bellman residual $B_\pi V - V$ (individual state components may be zero), where B_π is the Bellman expectation operator. Hence the TD update is a stochastic-approximation step toward solving $V = B_\pi V$. □

2.2 Tabular TD(0) for prediction

The TD(0) update is

$$V(s_t) \leftarrow V(s_t) + \alpha\, \delta_t. \tag{2.1}$$

Theorem 2.2.1 (Sutton 1988, Dayan 1992; convergence of TD(0))**.** *Assume (i) every state is visited infinitely often under π; (ii) step sizes satisfy the Robbins–Monro conditions $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$; (iii) rewards are bounded; (iv) the finite MDP is discounted with $\gamma < 1$. Then tabular TD(0) converges almost surely to V_π, with step-size conditions applied along each state's visits. On-policy linear TD needs additional feature and sampling assumptions and generally converges to a projected Bellman fixed point, not the exact V_π.*

Proof sketch. Apply the standard stochastic-approximation theorem to the operator B_π. The update (2.1) has the form $\theta_{t+1} = \theta_t + \alpha_t(H(\theta_t) + M_{t+1})$ where the drift is $H(V) = B_\pi V - V$. The Bellman operator B_π, rather than this difference, is a γ-contraction in ℓ_∞ and M_{t+1} is a martingale-difference noise with bounded variance. Tsitsiklis 1994 supplies the regularity conditions. □

2.3 n-step TD and the bias–variance dial

Define the n-step truncated return

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n V(s_{t+n}).$$

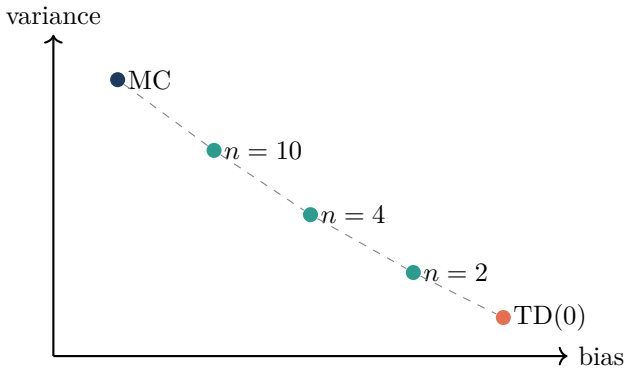


Figure 2.1: Schematic bias–variance trade-off for approximate values. This is not a universal Pareto frontier or a quantitative monotonicity claim.

Note $G_t^{(1)} = Y_t$ (the TD(0) target), and as $n \to \infty$ (or to the end of the episode) $G_t^{(n)} \to G_t$ (the Monte Carlo return). The n-step TD update is

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t^{(n)} - V(s_t)].$$

Proposition 2.3.1 (Bias–variance of n-step returns)**.** *Let the value function be exact ($V = V_\pi$). Then $G_t^{(n)}$ is an unbiased estimator of $V_\pi(s_t)$ for every n, and, for a fixed on-policy process with square-integrable return, $\mathrm{Var}[G_t^{(n)} \mid s_t]$ is non-decreasing in n.*

Proof sketch. Let $\mathcal{F}_{t+n}$ contain the trajectory through S_{t+n}, before the next action. With exact V^π, $G_t^{(n)} = \mathbb{E}[G_t \mid \mathcal{F}_{t+n}]$. These conditional expectations form a square-integrable martingale. The conditional-variance decomposition implies that revealing one more transition cannot decrease their variance conditional on S_t. With approximate values, this martingale argument no longer applies and neither bias nor variance need vary monotonically with n. □

2.4 SARSA: on-policy TD control

For action values the analogue update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)],$$

where $a_{t+1} \sim \pi(\cdot|s_{t+1})$ is the action the agent actually takes next. Hence the name SARSA = (state, action, reward, next-state, next-action).

Theorem 2.4.1 (Singh–Jaakkola–Littman–Szepesvári 2000)**.** *In a finite discounted MDP with bounded rewards, SARSA with GLIE behaviour (infinite visits to every state–action pair and greedy in the limit), and step sizes satisfying Robbins–Monro, converges almost surely to $Q^\star$ and the associated policy $\pi \to \pi^\star$.*

2.5 Q-learning: off-policy TD control

The Q-learning update replaces the bootstrap action with the greedy action:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)].$$

The target uses $\pi^\star$ regardless of the behaviour. This is the canonical off-policy control algorithm: the agent can learn $Q^\star$ from any exploratory behaviour as long as every (s, a) pair is visited infinitely often (Watkins–Dayan 1992).

Theorem 2.5.1 (Convergence of Q-learning, Watkins–Dayan 1992)**.** *In a finite discounted MDP with bounded rewards, per-state–action Robbins–Monro step sizes and the visitation assumption above, tabular Q-learning converges almost surely to* $Q^\star$.

2.6 Expected SARSA

Replace the sampled bootstrap with its expectation under the current behaviour policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma\, \mathbb{E}_{a' \sim \pi}[Q(s_{t+1}, a')] - Q(s_t, a_t)\big].$$

For a fixed value table and matching policy, integrating out the next action yields no greater target variance than SARSA by conditional variance, with equality possible (van Seijen et al. 2009). It is on-policy when the expectation matches the sampling policy and off-policy otherwise (if we evaluate against the greedy policy we recover Q-learning).

2.7 Maximisation bias and Double-Q

Proposition 2.7.1 (Maximisation bias of Q-learning)**.** *If* Q *is a noisy unbiased estimator of* $Q^\star$*, the operator* $\max_a Q$ *is a biased (in expectation, positively biased) estimator of* $\max_a Q^\star$.

Proof. max is convex; by Jensen's inequality, for any random variables X_a, $\mathbb{E}[\max_a X_a] \geq \max_a \mathbb{E}[X_a]$. □

Example 2.7.2 (Sutton & Barto 6.7)**.** Consider an MDP with two states $\{A, B\}$, A is start. From A there are two actions: *left* (to B, reward 0) and *right* (terminal, reward 0). From B there are k actions, each terminal, with rewards i.i.d. $\mathcal{N}(-0.1, 1)$. The optimal policy goes *right* (expected return $0 > -0.1$). Q-learning overestimates $\max_a Q(B, a)$ early in training and persistently chooses *left*. Double-Q reduces this mechanism; finite runs can still select the inferior branch.

Double-Q learning (Hasselt 2010) maintains two estimators Q_1, Q_2 updated on disjoint random halves of the data. On each step, with probability $\frac{1}{2}$

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha\big[r + \gamma Q_2(s', \arg\max_{a'} Q_1(s', a')) - Q_1(s, a)\big],$$

else the symmetric update. The argmax (selection) uses Q_1; the value (evaluation) uses Q_2. This reduces coupling between selection and evaluation; the learned tables need not be independent and the resulting target need not be exactly unbiased.

2.8 On-policy vs. off-policy: the cliff-jumping gap

For the same ε-greedy behaviour SARSA and Q-learning converge to different policies even in the limit $\alpha \to 0$. Q-learning's target is the greedy operator ($\pi^\star$, exploration-blind). SARSA's target is the current behaviour policy (exploration-aware: it factors in that ε fraction of the time we will slip).

On CliffWalking the greedy policy walks one square above the cliff (shortest); the ε-greedy version regularly falls in. SARSA learns this, prefers the longer safer path, and gets better online return at fixed ε. Q-learning maintains the greedy belief and pays for it during exploration.

Common pitfall

"Asymptotically equivalent if $\varepsilon \to 0$" is the standard textbook line but it lies about finite-time behaviour. In production any ε you actually deploy creates a SARSA vs. Q-learning behaviour gap that matters.

3 Algorithms

Algorithm 1: SARSA (on-policy TD control)

Input: step size α, discount γ, ε-schedule
Initialise $Q(s,a) = 0$ for all s, a;
for *episode = 1, 2, …* **do**
  $s \leftarrow$ `env.reset()`;
  $a \leftarrow \varepsilon$-greedy w.r.t. $Q(s,\cdot)$;
  while *not terminal* **do**
    $s', r, \text{done} \leftarrow$ `env.step`(a);
    $a' \leftarrow \varepsilon$-greedy w.r.t. $Q(s',\cdot)$ if not done else 0;
    $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a')\mathbb{1}[\neg\text{done}] - Q(s,a)]$;
    $s, a \leftarrow s', a'$;

Algorithm 2: Q-learning (off-policy TD control)

Input: step size α, discount γ, ε-schedule
Initialise $Q(s,a) = 0$ for all s, a;
for *episode = 1, 2, …* **do**
  $s \leftarrow$ `env.reset()`;
  while *not terminal* **do**
    $a \leftarrow \varepsilon$-greedy w.r.t. $Q(s,\cdot)$;
    $s', r, \text{done} \leftarrow$ `env.step`(a);
    $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a')\mathbb{1}[\neg\text{done}] - Q(s,a)]$;
    $s \leftarrow s'$;

Complexity. Both algorithms require $O(|\mathcal{S}||\mathcal{A}|)$ table storage, $O(1)$ additional storage per transition, and $O(|\mathcal{A}|)$ time per step (the per-step max). Total wall-clock per episode is $O(T \cdot |\mathcal{A}|)$ where T is episode length.

Example 3.0.1 (Worked numerical example)**.** A 1D corridor: states $\{0,1,2,3\}$, action $\in \{L, R\}$, reward -1 per step, $+10$ at state 3. Start at state 0, $\gamma = 1$. After one episode $(0, R, 1, R, 2, R, 3, +10)$ with $\alpha = 0.5$:

- SARSA target at $(s = 0, a = R)$: $-1 + Q(1, R) = -1 + 0 = -1$; update $Q(0, R) = 0.5(-1) = -0.5$.
- Q-learning target at $(s = 0, a = R)$: same here because $Q(1,\cdot) = 0$. The two diverge once Q has structure.

Common pitfall

Forgetting the `not done` mask: with $\gamma = 1$ and a terminal bonus reward, you will silently bootstrap from the post-terminal junk state and your value table will drift. This is the single most common bug in tabular TD code.

4 Implementation walkthrough

The reference implementations live in `project/agents.py`. The core SARSA loop:

```
def sarsa(env, n_episodes, alpha, gamma, eps_start, eps_end, eps_decay, seed):
    rng = np.random.default_rng(seed)
    nS, nA = env.observation_space.n, env.action_space.n
    Q = np.zeros((nS, nA))
    sched = make_eps_schedule(eps_start, eps_end, eps_decay)
    returns = []
    for ep in range(n_episodes):
        eps = sched(ep)
        s, _ = env.reset(seed=int(rng.integers(0, 2**31 - 1)))
        a = eps_greedy(Q, s, eps, rng)
        G, done, t = 0.0, False, 0
        while not done and t < max_steps:
            sn, r, term, trunc, _ = env.step(a)
            done = term or trunc
            an = eps_greedy(Q, sn, eps, rng) if not term else 0
            target = r + (0.0 if term else gamma * Q[sn, an])
            Q[s, a] += alpha * (target - Q[s, a])
            s, a = sn, an
            G += r; t += 1
        returns.append(G)
    return Q, returns
```

Q-learning differs by exactly two lines (select after observing s'; use $\max_{a'} Q(s', a')$ in the bootstrap). Expected SARSA replaces $Q(s', a')$ with $\sum_{a'} \pi(a'|s')Q(s', a')$.

Common pitfall

In gymnasium 1.x, `env.reset()` returns the tuple `(obs, info)` and `env.step()` returns 5-tuples `(obs, reward, terminated, truncated, info)`. Confusing `terminated` (true terminal) with `truncated` (time-limit) is critical: never bootstrap through a true terminal; do bootstrap through a truncation. Use either flag to end collection, but only true termination removes the bootstrap for an external time limit. A genuine finite-horizon task instead includes remaining time in its state and terminates at the horizon.

5 Empirical results

5.1 CliffWalking bake-off

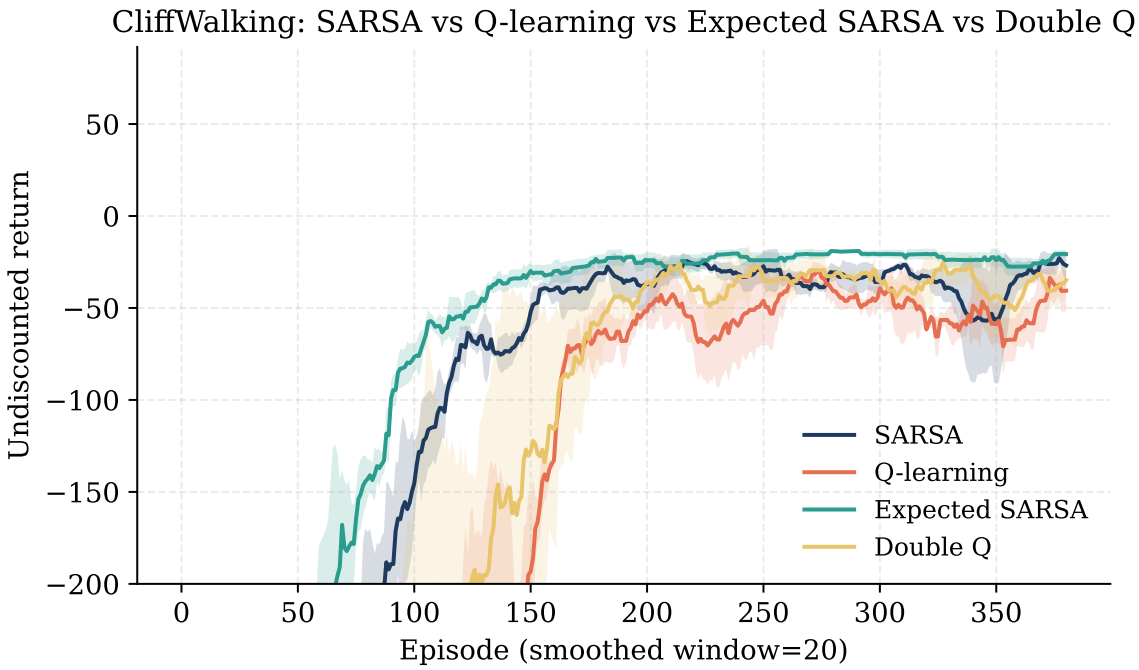


Figure 5.1: CliffWalking learning curves (mean ±1 std across 3 seeds, smoothed window 20). SARSA and Expected SARSA quickly settle on the safer longer path. Q-learning and Double-Q learn the optimal greedy path but pay an exploration cost: a random exploratory action near the cliff can cause a fall, although many exploratory actions remain safe.

Bias–variance dial:

5.2 Taxi-v3

Safety angle

The cliff is literal in production. Q-learning's optimism near the cliff is not a curiosity but the prototype of a real safety failure mode: an off-policy agent commits to the greedy policy and ignores the cost of having to explore around it. In a high-stakes domain (an industrial robot, a credit underwriter, a clinical decision-support system) any deployed exploration — ε, NoisyNet, entropy bonus — carries asymmetric downside. Two concrete mitigations:

1. **Use on-policy or Expected SARSA when you cannot remove ε at deployment.** The policy you evaluate against is the policy you will run.

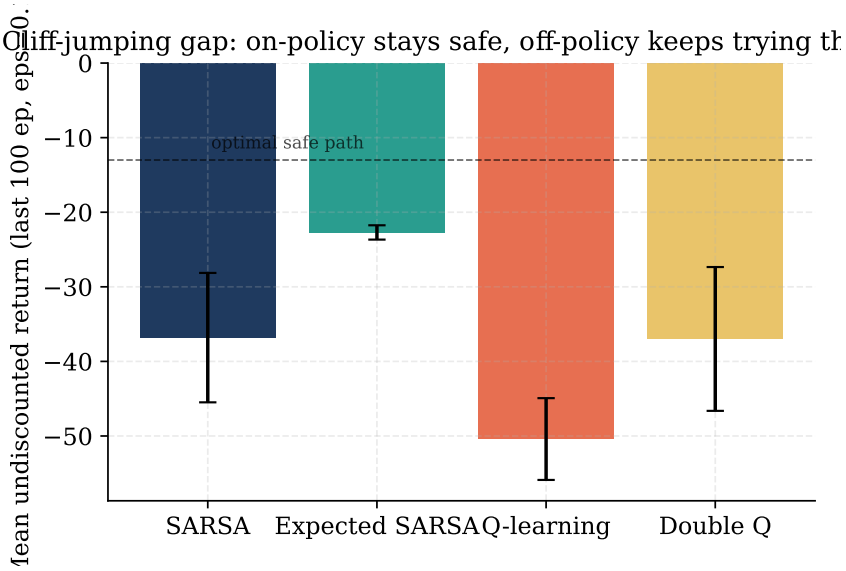


Figure 5.2: The cliff-jumping behaviour gap: mean undiscounted return over the last 100 episodes at $\varepsilon = 0.1$. SARSA/Expected-SARSA cluster near the safe-path optimum; Q-learning sits noticeably worse because its ε-greedy rollouts keep falling.

2. **Bound the exploratory action set.** A shield or action mask (Alshiekh et al., AAAI 2018, see `research/safety_rl_sota.md` §2) eliminates the cliff-adjacent action outright. This can prevent the specified hazard if the model and admissible-action set are correct; it does not certify unmodelled hazards.

These themes recur in Weeks 22–23 (CMDPs and shielded RL). For now: if your bandit/RL policy's behaviour gap between training and serving has a cost asymmetry, prefer SARSA or Expected SARSA over Q-learning.

Matching application

Lead-scoring as TD control. A synthetic insurance-style lead-routing policy can be cast as a tabular MDP: state = (lead segment, broker capacity bucket), action = which broker to assign, reward = signed margin of the conversion outcome. Per-conversion online updates implement TD(0):

$$Q(\text{segment}, \text{broker}) \leftarrow Q(\text{segment}, \text{broker}) + \alpha \left[\text{realised margin} - Q(\text{segment}, \text{broker})\right],$$

i.e. contextual-bandit-style; the $\gamma Q(s', a')$ term collapses because the next lead is independent of the current routing decision (no carry-over state). Expected SARSA is the natural choice once you want to roll out a target policy π_t that is not the deployed π_b: you bootstrap against $\mathbb{E}_{a' \sim \pi_t}[Q(s', a')]$ while collecting data under π_b, exactly the off-policy structure we exploit in Week 11 with SNIPS and DR. Exercise 9 of the workbook walks through the SNIPS estimator for a counterfactual reward of swapping π_b for π_t on a logged trajectory — the first taste of off-policy evaluation in this curriculum.

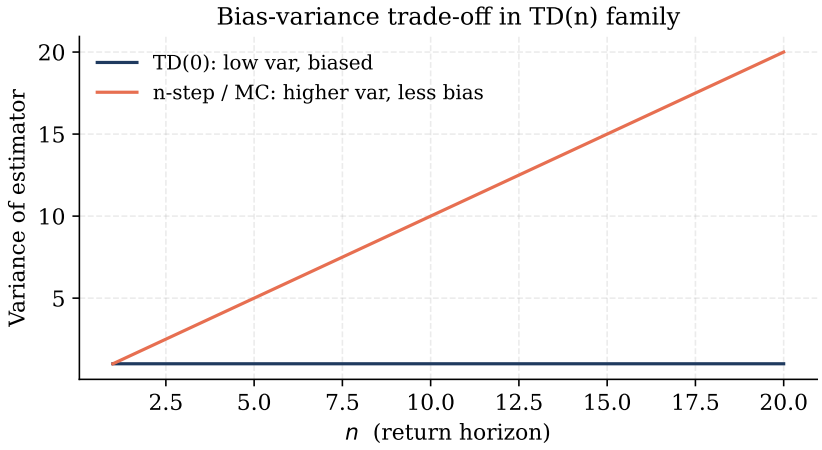


Figure 5.3: Schematic variance of the n-step return target. For a fixed (correct) value function, target variance is non-decreasing under the assumptions of Proposition 2.3.1; it need not grow linearly. An imperfect V removes that variance ordering. These curves are illustrative.

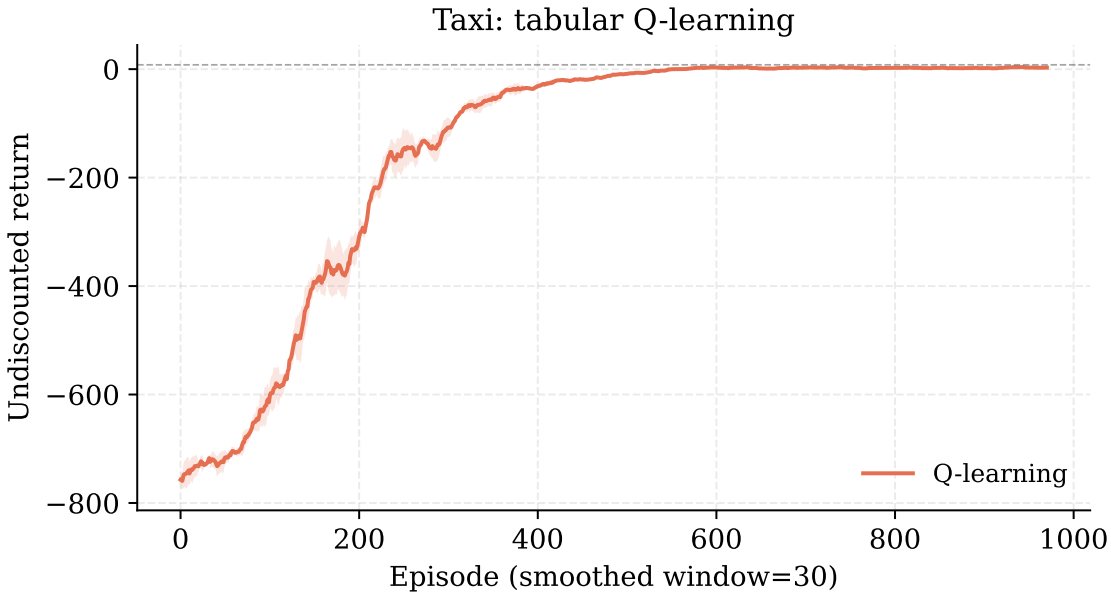


Figure 5.4: Tabular Q-learning on Taxi-v3, 1,000 episodes, $\alpha = 0.5$, $\gamma = 0.99$. Reaches mean return well above the random-policy baseline (≈ -800).

6 Exercises

The full graded notebook is in `workbook.ipynb`. Summary:

1. TD(0) prediction update (★).
2. n-step return computation (★★).
3. SARSA single-step update (★).
4. Q-learning single-step update (★).
5. Expected SARSA bootstrap target (★★).
6. Double-Q step (★★).
7. Empirical TD vs MC variance (★★).
8. CliffWalking bake-off (★★).
9. SNIPS off-policy evaluator for a lead-router (★★★).

Intuition. For Exercise 9, remember that SNIPS divides by $\sum_i w_i$ rather than N: the self-normalisation changes finite-sample bias and variance. Under suitable moment assumptions the bias is $O(1/N)$ and vanishes asymptotically; variance or MSE superiority over IPS is not universal.

7 Further reading

The companion `extended_reading.pdf` covers Sutton 1988 (original TD), Watkins & Dayan 1992 (Q-learning convergence), van Hasselt 2010 (Double-Q), van Seijen et al. 2009 (Expected SARSA), and the historical Sutton & Barto Chapter 6. Foundational material that will be cited again in Weeks 5–6 (DQN family) and Week 11 (off-policy evaluation).